# What is Java?

Java is a **programming language** and a **platform**. Java is a high-level, robust, object-oriented and secure programming language.

Java was developed by *Sun Microsystems* (which is now a subsidiary of Oracle) in the year 1995. *James Gosling* is known as the father of Java. Before Java, its name was *Oak*. Since Oak was already a registered company, so James Gosling and his team changed the name from Oak to Java.

# Features of Java:

A list of the most important features of the Java language is given below.

1. Simple
2. Object-Oriented
3. Portable
4. Platform independent
5. Secured
6. Robust
7. Architecture neutral
8. Interpreted
9. High Performance
10. Multithreaded
11. Distributed
12. Dynamic

# Object-oriented

Java is an object-oriented programming language. Everything in Java is an object. Object-oriented means we organize our software as a combination of different types of objects that incorporate both data and behavior.

Object-oriented programming (OOPs) is a methodology that simplifies software development and maintenance by providing some rules.

Basic concepts of OOPs are:

1. Object
2. Class
3. Inheritance
4. Polymorphism
5. Abstraction
6. Encapsulation

## Difference Between C++ and Java:

| Feature | Java | C++ |
|---|---|---|
| Platform | Runs on any device with JVM (Write Once, Run Anywhere) | Runs directly on operating system (needs recompiling) |
| Memory Management | Automatic (Garbage Collector handles it) | Manual (Programmer manages memory) |
| Ease of Use | Simpler and safer (no pointers) | More complex (uses pointers and manual memory) |
| Performance | Slower (runs on JVM) | Faster (runs directly on machine) |
| Use Cases | Web apps, Android apps, enterprise software | Games, system software, real-time applications |
| Programming Style | Purely Object-Oriented | Supports both Procedural and Object-Oriented |
| Security | More secure (runs in a virtual environment) | Less secure (more direct access to system memory) |

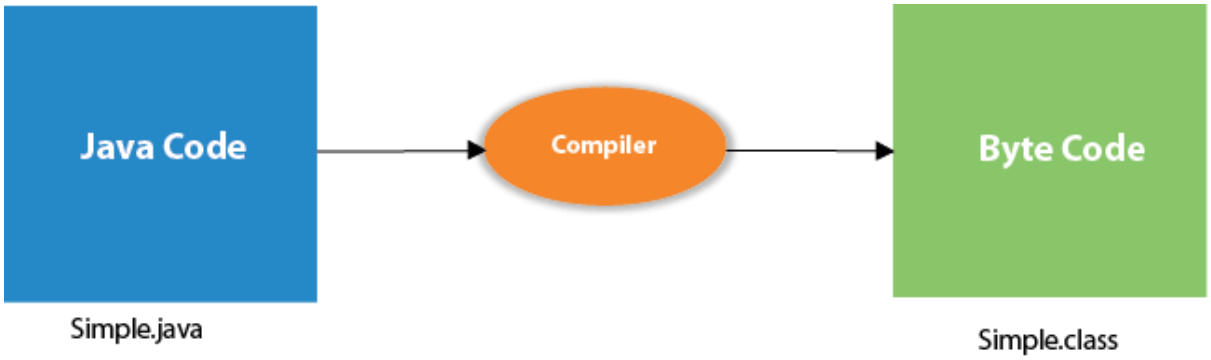
## Hello World - Java Program:

Example

```
1. public class Main
2. {
3.  public static void main(String args[])
4. {
5.   System.out.println("Hello World!");
6.  }
7. }
```

# What happens at compile time?

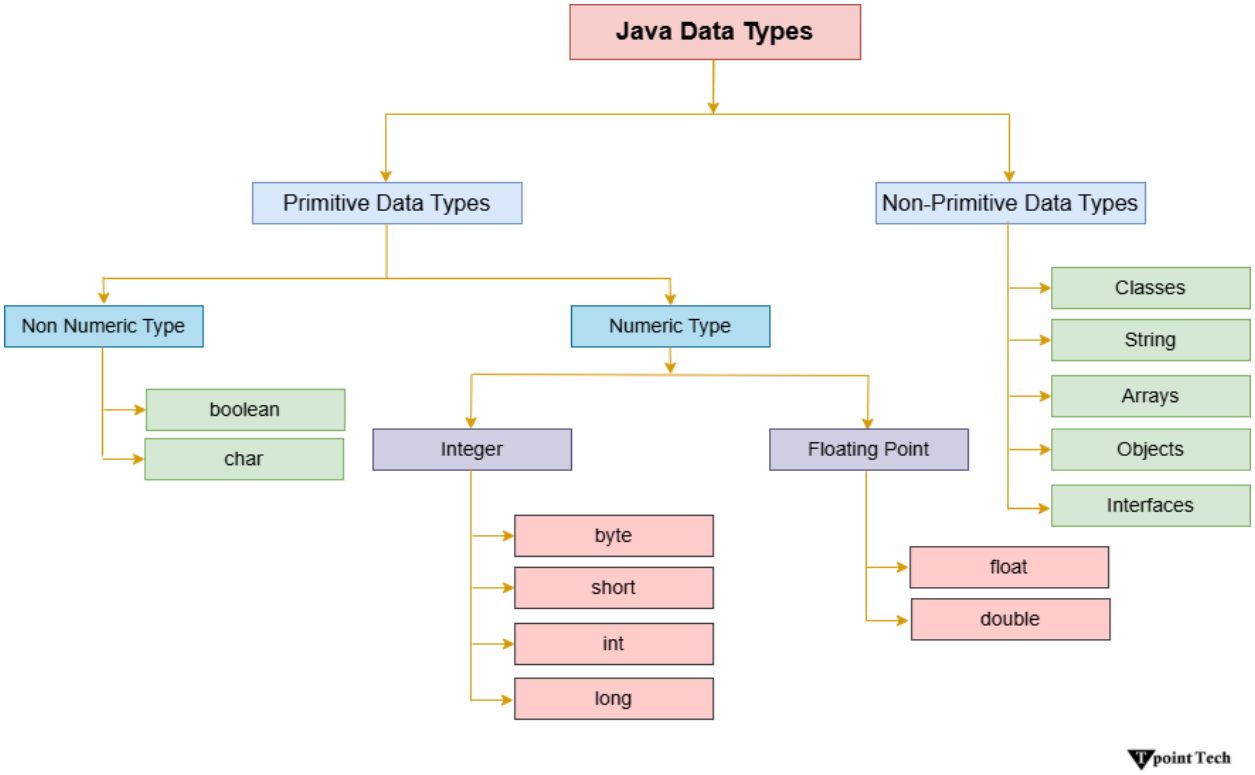
At compile time, the Java file is compiled by Java compiler (It does not interact with OS) and converts the Java code into bytecode.



Data Types in Java:

In programming languages, data types specify the different sizes and values that can be stored in the variable or constants. Each data type is predefined, which makes Java a statically and strongly typed language. There are the following two types of data types in Java.

1. **Primitive Data Types:** The primitive data types include boolean, char, byte, short, int, long, float and double.
2. **Non-Primitive Data Types:** The non-primitive data types include Classes, Interfaces, String, and Arrays.




Java Variable:

A variable is a container which holds the value while the Java program is executed. A variable is assigned with a data type.

Variable is a name of memory location. There are three types of variables in java: local, instance and static.

A variable declared inside the body of the method is called local variable. You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.

1. int data=50;*//Here data is variable*

```
int data;
```

**Operators** are an essential part of any programming language. In Java, operator is a symbol that is used to perform operations. For example: +, -, *, / etc. These are essential for performing different types of operations on variables and values. In this section, we will discuss different types of operators used in Java programming.

There are mainly eight types of operators in Java:

1. Unary Operator
2. Arithmetic Operator
3. Relational Operator
4. Ternary Operator
5. Assignment Operator
6. Logical Operator

The Java unary operators require only one operand. Unary operators are used to perform various operations i.e. incrementing/decrementing a value by one, negating an expression and inverting the value of a Boolean. Observe the following table.

| Operator | Description | Example | Meaning |
|---|---|---|---|
| + | Unary plus (usually redundant) | +a | Indicates positive value |
| - | Unary minus | -a | Negates the value (makes positive → negative) |
| ++ | Increment operator | ++a or a++ | Increases value by 1 |
| -- | Decrement operator | --a or a-- | Decreases value by 1 |
| ! | Logical NOT | !true → false | Reverses a boolean value |

Arithmetic operators:

| Operator | Name | Description |
|---|---|---|
| + | Addition | Adds two numbers |
| - | Subtraction | Subtracts second number from first |
| * | Multiplication | Multiplies two numbers |

| Operator | Name | Description |
|---|---|---|
| / | Division | Divides first number by second |
| % | Modulus (Remainder) | Returns remainder after division |

```
public class ArithmeticExample {
   public static void main(String[] args) {
      int a = 10, b = 3;

      System.out.println("Addition: " + (a + b));
      System.out.println("Subtraction: " + (a - b));
      System.out.println("Multiplication: " + (a * b));
      System.out.println("Division: " + (a / b));
      System.out.println("Modulus: " + (a % b));
   }
}
```

Java Relational Operators

Java relational or conditional operators are used to check relationship between two operands such as less than, less than or equal to, greater than, greater than or equal to, equal to and not equal to. It is also known as equality operator. These operators return Boolean values either true or false.

| Operator | Name | Description | Example | Result |
|---|---|---|---|---|
| == | Equal to | Checks if two values are equal | a == b | true if equal |
| != | Not equal to | Checks if two values are not equal | a != b | true if not equal |
| > | Greater than | Checks if left value is greater | a > b | true if a > b |
| < | Less than | Checks if left value is smaller | a < b | true if a < b |
| >= | Greater than or equal | Checks if left value is greater or equal | a >= b | true if a ≥ b |

| Operator | Name | Description | Example | Result |
|---|---|---|---|---|
| <= | Less than or equal | Checks if left value is smaller or equal | a <= b | true if a ≤ b |

Assignment Operator

There are two types of assignment operators. They are:

- **Simple Assignment operator:** The simple assignment operator where only (=) is used.
- **Compound Assignment operator:** The compound assignment operator is used where -,+,/,* are used along with (=) operator.

| Operator | Description | Example | Same As |
|---|---|---|---|
| = | Assign | a = 5 | a = 5 |
| += | Add and assign | a += 3 | a = a + 3 |
| -= | Subtract and assign | a -= 2 | a = a - 2 |
| *= | Multiply and assign | a *= 4 | a = a * 4 |
| /= | Divide and assign | a /= 2 | a = a / 2 |
| %= | Modulus and assign | a %= 3 | a = a % 3 |

```
public class Main {
public static void main(String[] args) {
int a=10;
a+=3;//10+3
System.out.println(a);
a-=4;//13-4
System.out.println(a);
a*=2;//9*2
```

```
System.out.println(a);
a/=2;//18/2
System.out.println(a);
a %= 3; // 9 % 3 = 0
System.out.println(a);
}}
```

Logical Operators

Logical operators are used extensively in various programming languages to perform Logical NOT, OR and AND operations whose functionality is similar to OR gate and AND gate in the world of digital electronics.

| Operator | Name | Description | Example | Result |
|---|---|---|---|---|
| && | Logical AND | Returns true if **both** conditions are true | (a > 5 && b < 10) | true if both are true |
| \|\| | Logical or | Returns true if **at least one** condition is true | Logical OR | Returns true if **at least one** condition is true |
| ! | Logical NOT | Reverses the boolean value | !(a > 5) | true if a ≤ 5 |

```
public class LogicalExample {
   public static void main(String[] args) {
      int a = 7, b = 3;

      System.out.println("a > 5 && b < 10: " + (a > 5 && b < 10)); // true
      System.out.println("a < 5 || b < 10: " + (a < 5 || b < 10)); // true
      System.out.println("!(a == b): " + !(a == b));             // true
   }
}
```

&&

T F=F

F T=F

F F=F

||

T T=T

T F=T

T T=T

F F =F